

4. Счётчик над или под списком: «Осталось: N, Выполнено: M».
5. Фильтр из трёх кнопок/вкладок: «Все», «Активные», «Завершённые». При клике на фильтр список визуально показывает только соответствующие задачи (остальные скрываются, например через `display: none`, а не удаляются из данных).

Технические требования (обязательные):

- Вся логика — в одном файле `script.js`, подключённом к `task3/index.html` через `<script src="script.js" defer></script>`.
- HTML-разметка отдельных задач **генерируется в JavaScript** (через `document.createElement` или сборку строки и `innerHTML`), не прописывается заранее вручную в HTML.
- Список задач должен храниться в JS в виде массива объектов, например:

```
// { id: 1, text: "Сделать лабу", completed: false }
```

Отрисовка списка на экране должна происходить на основе этого массива (функция `render()`, которая перерисовывает список при любом изменении данных).

- Использовать методы массивов `map` / `filter` / `forEach` там, где это уместно (например, `filter` — для отображения по текущему фильтру, `map` — при рендере).
- Каждой задаче должен присваиваться уникальный идентификатор (можно просто счётчик, увеличивающийся на 1, или `Date.now()`).
- Обработка событий — через `addEventListener` (не через атрибуты `onclick="..."` в HTML).
- В консоли браузера (F12) не должно быть ни одной ошибки при использовании приложения (добавление, удаление, отметка выполнения, переключение фильтров).

Что сдавать: `task3/index.html`, `task3/style.css`, `task3/script.js`.

Критерии проверки: все 5 пунктов функциональности работают, данные хранятся в массиве и рендерятся из него, нет `onclick` в HTML, нет ошибок в консоли.

Лабораторная работа №4. Игра «Быки и коровы»

Цель: закрепить работу с состоянием приложения, массивами/строками и условной логикой без использования Canvas.

Правила игры (если незнакомы): компьютер загадывает число из 4 неповторяющихся цифр (от 0 до 9). Игрок вводит своё предположение — тоже 4 цифры. После каждой попытки показывается результат: количество «быков» (цифра угадана и стоит на своём месте) и «коров» (цифра есть в загаданном числе, но стоит на другом месте).

Что нужно сделать:

1. При загрузке страницы (или по кнопке «Новая игра») генерируется случайное 4-значное число с неповторяющимися цифрами.
2. Поле ввода для попытки игрока + кнопка «Проверить».

3. **Валидация ввода:** ровно 4 цифры, все цифры разные, только цифры (без букв/символов).
При некорректном вводе — сообщение об ошибке, попытка не засчитывается.
4. После каждой попытки в списке (историю) добавляется строка вида: 1234 → 1 бык, 2 коровы.
5. Счётчик количества попыток.
6. Проверка на победу: если «быков» равно 4 — игра завершена, показывается сообщение «Победа! Угадано за N попыток» и поле ввода блокируется до нажатия «Новая игра».
7. Кнопка «Новая игра» — сбрасывает состояние (загаданное число, историю попыток, счётчик).

Технические требования (обязательные):

- Логика генерации числа, сравнения и подсчёта быков/коров — написать самостоятельно (без готовых прт-библиотек).
- Использовать функции: отдельная функция для генерации числа, отдельная — для валидации ввода, отдельная — для подсчёта быков и коров. Не писать всё одним большим обработчиком клика.
- Файлы: `task4/index.html`, `task4/style.css`, `task4/script.js` (Canvas в этой лабораторной не нужен, обычный DOM).
- История попыток должна рендериться из массива данных (как в лабораторной №3), а не добавляться напрямую в `innerHTML` без хранения состояния.

Что сдавать: папку `task4/`.

Критерии проверки: корректность подсчёта быков/коров на разных наборах входных данных (преподаватель проверит вручную несколькими комбинациями), валидация ввода работает, есть возможность сыграть заново без перезагрузки страницы.

Лабораторная работа №5. Игра Simon (Симон говорит)

Цель: освоить работу с таймерами/задержками, звуком (по желанию) и последовательностями событий.

Референс: можно ориентироваться на freesimon.org, но верстка/цвета — свои, не копировать 1-в-1 чужой код.

Правила игры: на экране 4 цветные кнопки/сектора. Компьютер «показывает» последовательность (по очереди подсвечивает кнопки с паузой между ними). Игрок должен повторить последовательность в том же порядке, нажимая на кнопки. Если повторил верно — добавляется ещё один шаг к последовательности, и раунд повторяется заново (с начала, с уже более длинной последовательностью). Если ошибся — игра заканчивается, показывается достигнутый результат (длина последовательности).

Что нужно сделать: